Wafer Mapping Tool — Instructional Document

Introduction

This document explains **what the wafer-mapping reliability tool does and how to use it**. It is the successor to the original "single-structure" proof of concept. If you used the older flow (the one driven by `run_engine_physical.py` and `Global_Params.py`), this guide will feel familiar in spirit, but almost everything underneath has been rebuilt and expanded.

As before, the tool predicts the **time-to-failure (TTF)** behaviour of a wafer under the assumption that **each cell of the wafer contains a single test structure** with negligible line area. That structure is, as before, referred to as **TSK-Alpha**.

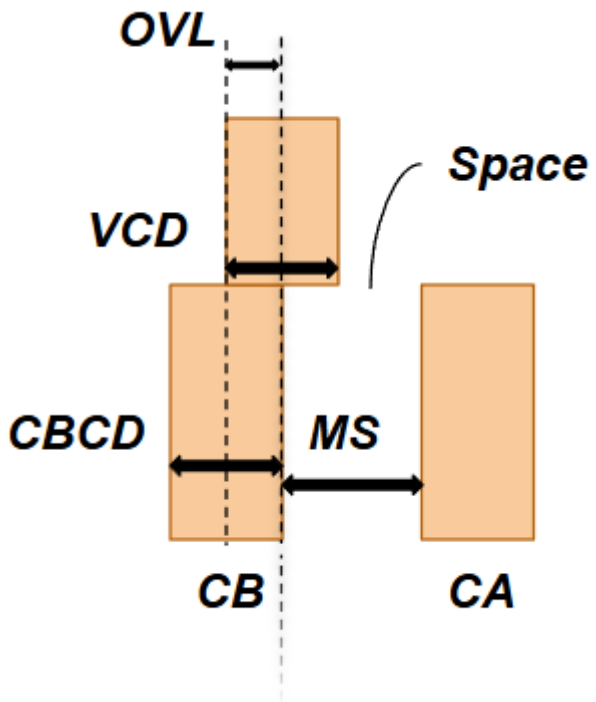


Figure 1: Illustration of TSK-Alpha Structure.

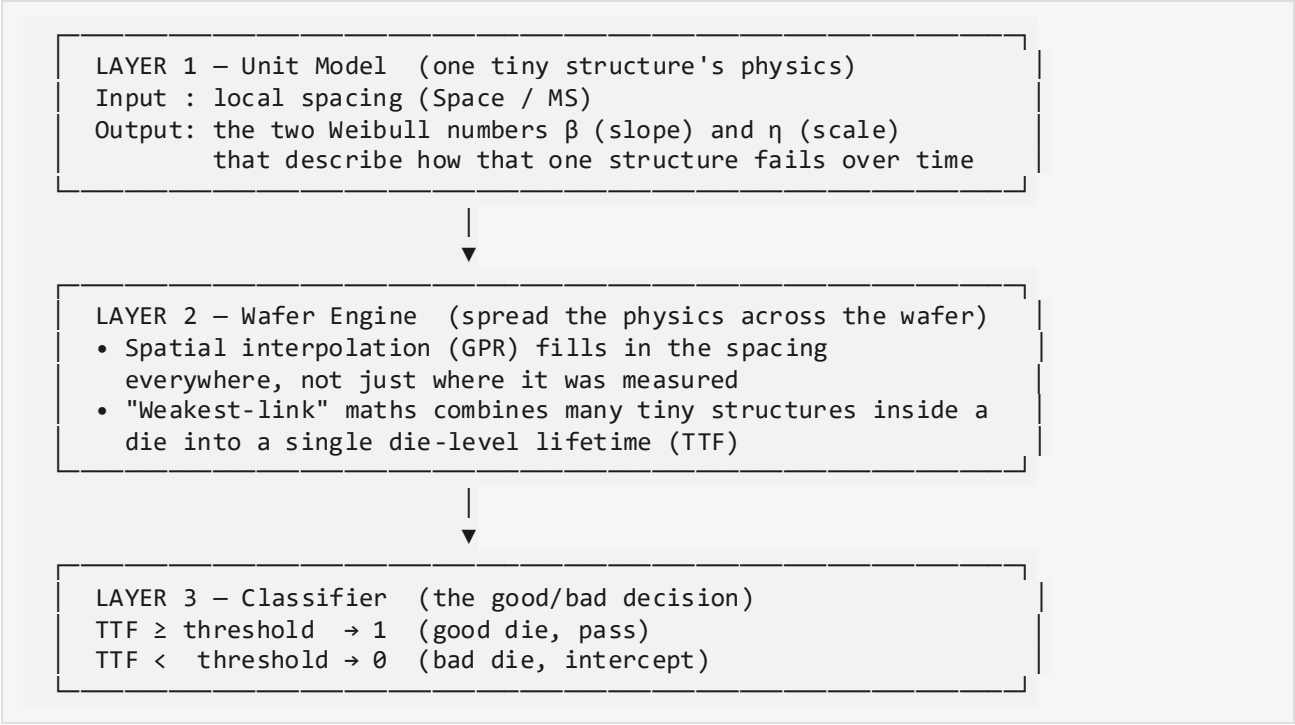
While some fundamentals are preserved, the new flow expands its scope. The old tool took a wafer's spacing measurements and produced reliability maps directly. The new "Full Engine" does three things in sequence, and it can also **learn its own settings from labelled data**:

1. **Predict:** a continuous lifetime (TTF) for every die on the wafer.
2. **Classify:** every die as **good** or **bad** using a decision threshold.
3. **Self Tune:** given wafers whose true good/bad outcome is known, the engine automatically searches for the settings that best reproduce those outcomes. As of this revision it can also **re-calibrate the physics itself** to your process (more in "*Delta training*" section).

In short: the old tool made predictions based on explicitly defined simulated or empirically obtained relationships. The new tool uses these as a base but integrates an automatic tuning loop in addition to threshold based classifier. The rest of this document walks through the models it offers, the data it expects, the files it produces, and exactly how to run and adjust it.

The Big Picture: a three-layer pipeline

Everything the engine does is organised into three stacked layers. This explanation is provided for the sake of understanding and documentation. It is important to note that an operator does NOT have to interact with them individually but understanding them provides useful context.



In summary: **Layer 1** is the “unit expert” which knows how one unit fails. **Layer 2** attempts to represent the contents of the die as a collection of our representative structures given a limited spatial measurement, and accounts for weakest link failure. **Layer 3**, empowered by the first two layers, can determine if the reliability threshold is met.

The three values that control Layers 1–2 (`M_structures`, `F_target`, `N_samples_per_dim`) and the one that controls Layer 3 (`threshold`) are explained in the “*How to Adjust*” section. The function of training is predominantly to extract these figures and, optionally, to fine-tune the physics equations inside Layer 1 as well. This process takes place automatically given limited measurement and labeled data marking the most significant improvement beyond the previous pipeline.

Models

The tool supports three families of **unit models** (Layer 1). Only one unit model is to be used at a time but multiple are in included for the sake of comparison. They all serve the same fundamental role of answering *"given the spacing here, what are the Weibull failure parameters?"* however, they do this differently.

Physically Based models (DPM / IMEC / Empirical)

These are the direct descendants of the old flow's physics engine. They derive their form from classical, physically based TDDB models:

- **Power Law** (`PowerLaw`)
- **Square-root E** (`SqrtE`)
- **Inverse E / 1-E** (`InverseE`)

They combine empirically fit, spacing-dependent parameters with a simulation-based backbone (DPM). This family:

- assumes a defect size of 0.45 nm
- uses the **SSPP approximation** for via-line breakdown,
- produces predictions in **real units** (e.g. seconds or years)
- predicts **both** via-line **and** line-line breakdown.

The empirical data each is based on was collected by **IMEC**. It is worth repeating that although the model is physically and empirically rooted, **for best results the fit should be redone for your specific process / dielectric**. The current fits reflect IMEC's process but not necessarily others. Because these models are process specific, they can fail when applied to differing domains. This possibility was expectedly reflected in the misalignment that occurrent when applying these models to our simulated data. While the best solution is refitting, the two mechanisms below (spacing scaling (older) and delta training (new)) are tools introduced to close this gap.

> **Operating-voltage note:** These models use an operating voltage `V_OP`, which defaults to **0.7 V** in the code. Electric field is computed locally as `E = V_OP / spacing`, so voltage and spacing always travel together.

Scaled DPM variants — proof of concept fix for domain mismatch

The DPM constants were calibrated in a way such that their discriminatory power is best for sub-5 nm geometries. The fabricated dataset in this project has spacings predominantly in the **9–22 nm** range. When you feed wide spacings into equations tuned for narrow ones, every die looks almost equally reliable, the predicted lifetimes bunch together, and the good/bad threshold has nothing to grab onto.

The first fix was a **spacing scale factor** `k`. Before any physics is computed, the measured spacing `S` is divided by `k`:

```
S_effective = S_measured / k
```

This re-centres real-world spacings into the range where the equations were calibrated. It **does not** changing any of the physics coefficients. For a 9–22 nm dataset a good starting point is **`k = 3.0`**:

```
  9 nm  →  3.0 nm   (high-field, short-lifetime regime)
 15 nm  →  5.0 nm   (calibration boundary)
 22 nm  →  7.3 nm   (moderate-field regime)
```

Setting `k = 1.0` reproduces the original, unscaled behaviour exactly. The scaled models are named with a `Scaled` suffix:

- `PowerLawScaled`, `SqrtEScaled`, `InverseEScaled`

The value of `k` can be chosen with the command line using `--spacing-scale`. **Spacing scaling is a blunt, one-parameter "stretch"** as it slides the whole curve sideways but cannot change its shape otherwise. It can be well thought of as a realignment of the data rather than any sort of recalibration. Delta training (next) is the more powerful successor.

Delta-trained DPM variants — fine tuning to the target process

Spacing scaling has exactly **one knob** (`k`). Delta training expands with a small set of trainable corrections (deltas) placed directly inside the physics equations. Where scaling can only shift the curve, deltas reshape it, so the same physics can be fitted to a process it was never calibrated for.

Each empirical, spacing-dependent sub equation in the DPM model carries two additive corrections. The optimiser is vary them (within safe limits) to make the model's predictions match labelled data:

Sub-equation (what it governs)	**Its deltas**	**Shared or model-specific**
Weibull-slope coefficient `m(S)`	`m_val_delta_1`, `m_val_delta_2`	Shared by all three DPM models
Characteristic field `E_s(S)`	`Es_val_delta_1`, `Es_val_delta_2`	Shared (1/E model tightens one bound)
Field-acceleration factor	`ma_*` (PowerLaw) / `gamma_*` (SqrtE) / `G_*` (InverseE)	One pair per model

Features of Delta Training:

- **Every delta defaults to `0.0`. At `0.0` the model is identical to the original IMEC fit.** Turning delta training off changes nothing. Existing config with no deltas behaves exactly as before.
- **Deltas only touch the *empirical fits*, not the *simulation backbone*.** The percolation-simulation constants (`C_sc`, `P_sc`, `W_mu`, `mu_sc`) are deliberately unaltered. This ensures optimization is calibrating the fitted curve. It leaves the theoretical percolation theory alone.
- **Each delta is limited by physically justified bounds.** The allowed range of every delta is fixed for singularity avoidance, sign-flip prevention, pole control for the 1/E model. The optimiser cannot wander outside those rails.
- **Trained deltas ride inside the config.** Delta params are stored in `unit_model_kwargs["deltas"]`, so a trained `config.json` is available to `inference.py` automatically. This enable seamless integration into the previously established pipeline.

> ⚠ **Read this before trusting a delta score.** In the *current* dataset, `ExistenceClass` is defined **purely by spacing buckets** (see Inputs). On this data, a good score can only mean "the model made TTF increase smoothly with spacing," **not** "the physics is now calibrated to real reliability." The fitted delta numbers are a valid demonstration that the *trainer works*, but they are disposable until the next team supplies **real reliability labels or measured TTFs**.

To turn delta training on, add `--train-deltas` at training time.

Statistically / ML-Based model (GPR)

Rather than running a raw percolation simulation live, the new engine uses a **Gaussian Process Regression (GPR)** model that has been **trained on pre-computed simulation data**.

Percolation simulations are performed ahead of time with the resulting Weibull parameters across a grid of spacings recorded. To obtain additional spacings, GPR model instantly returns the interpolated Weibull β and η, no live simulation required. The model is selected simply as:

- `GPR`

The pre-computed simulation files live in the repository under:

```
src/unit_level_models/weibull_gpr_model_simulated_data/
  └─ vy10.50_vz21.00_lx10.50_ly21.00_lz21.00_r0.45/   ← defect radius 0.45 (default)
  └─ vy10.50_vz21.00_lx10.50_ly21.00_lz21.00_r0.65/   ← defect radius 0.65
```

Two user dictated params select which simulation surface is loaded:

Knob	Meaning	Valid values
`m_param`	Weibull modulus of the underlying simulation set	1.0, 2.0, … 10.0
`radius`	Modelled defect radius	0.45 (default) or 0.65

> The old purely-statistical percolation engine returned failure times in abstract "steps" (comparative, not real units). The GPR surrogate used here is wrapped to return the same kind of Weibull parameters as the physics models. This allows it to slot into the exact same Layer 2 / Layer 3 machinery. It still does not model electrical effects such as operating voltage. That only exists in the physics models. GPR also has no deltas. This means "`--train-deltas`" does nothing.

The Linear baseline

New in this revision is a deliberately simplistic model. It is selected with `--pipeline-type Linear`. It throws away all the physics. It ignores Weibull parameters, GPR interpolation, the weakest-link, and simply defines each die's TTF proxy as

```
TTF_proxy = VL_spacing + LL_spacing
```

This is included only for comparison. If the elaborate DPM/GPR pipeline cannot beat it, the physics is adding no predictive value on that data. A good model should sit above the Linear. `M_structures`, `F_target`, and `N_samples_per_dim` are accepted for interface compatibility but have no effect on Linear predictions.

Overview of Models

Family	Models	Units	Models V-L and L-L?	Voltage-aware?	Adaptable?
Physics (DPM)	PowerLaw, SqrtE, InverseE	Real (s / yr)	Yes (separately)	Yes	**Yes. Uses `--train-deltas`** .
Physics scaled	Previous Scaled (×3)	Real (rescaled)	Yes	Yes	Technically. Blunt "k" scaling.
ML surrogate	GPR	Simulation-derived	Combined (joint)	No	No. Does not need, works out of the box
Linear baseline	Linear	Spacing (since it's a sum)	Combined (raw)	No	No

Currently the **delta-trained physics models are now the strongest performers**, edging out GPR. This is a result that should be considered cautiously due to the domain we have tested it in however these models are promising. They provide physical interpretability, adjust automatically, and are based on empirical data. **Unscaled** physics models fit poorly out of the box, but this is due to (the geometry mismatch described above. The **scaled** physics models are a usable middle ground however they are

only introduced as a proof of concept. The **delta-trained** models are the recommended endpoint. See Pre-trained configs for measured scores.

Inputs

Dataset layout

The engine expects a strict directory structure. Training, validation, and test data all follow the **same** pattern:

```
<dataset_root>/
├── lot_001/
│   └── csv/
│       ├── wafer_01/
│       │   ├── Space.csv            ← V-L spacing measurement (2-D grid)
│       │   ├── MS.csv               ← L-L / metal-stack measurement (2-D grid)
│       │   └── ExistenceClass.csv   ← Ground-truth good/bad labels (2-D grid)
│       ├── wafer_02/
│       │   └── ...
│       └── ...
├── lot_002/
│   └── ...
└── ...
```

The engine looks for the pattern `lot_*/csv/wafer_*` exactly. A wafer folder that is nested any other way will not be found.

The three input files

Every wafer folder contains three CSVs, each a comma-separated 2-D grid of the **same shape** (the grid mirrors the physical layout of dies on the wafer):

File	What it measures	Used for
`Space.csv`	**V-L spacing** (the "via" spacing)	Via-line TDDB prediction
`MS.csv`	**L-L spacing** (the "line" spacing)	Line-line TDDB prediction
`ExistenceClass.csv`	“Ground truth” class of each die	Training labels & scoring

Existence Conventions:

- **Background cells.** A value that is `0`, `NaN`, or `Inf` is treated as if there is no die there, and the cell is ignored.
- **Label encoding** in `ExistenceClass.csv`:

Raw value	Meaning	Internal label
`1` or `2`	bad die	`0` (intercept)
`3`	good die	`1` (pass)

At the die level, the engine predicts that internal `0/1` label (and the lifetime behind it) from the two spacing maps.

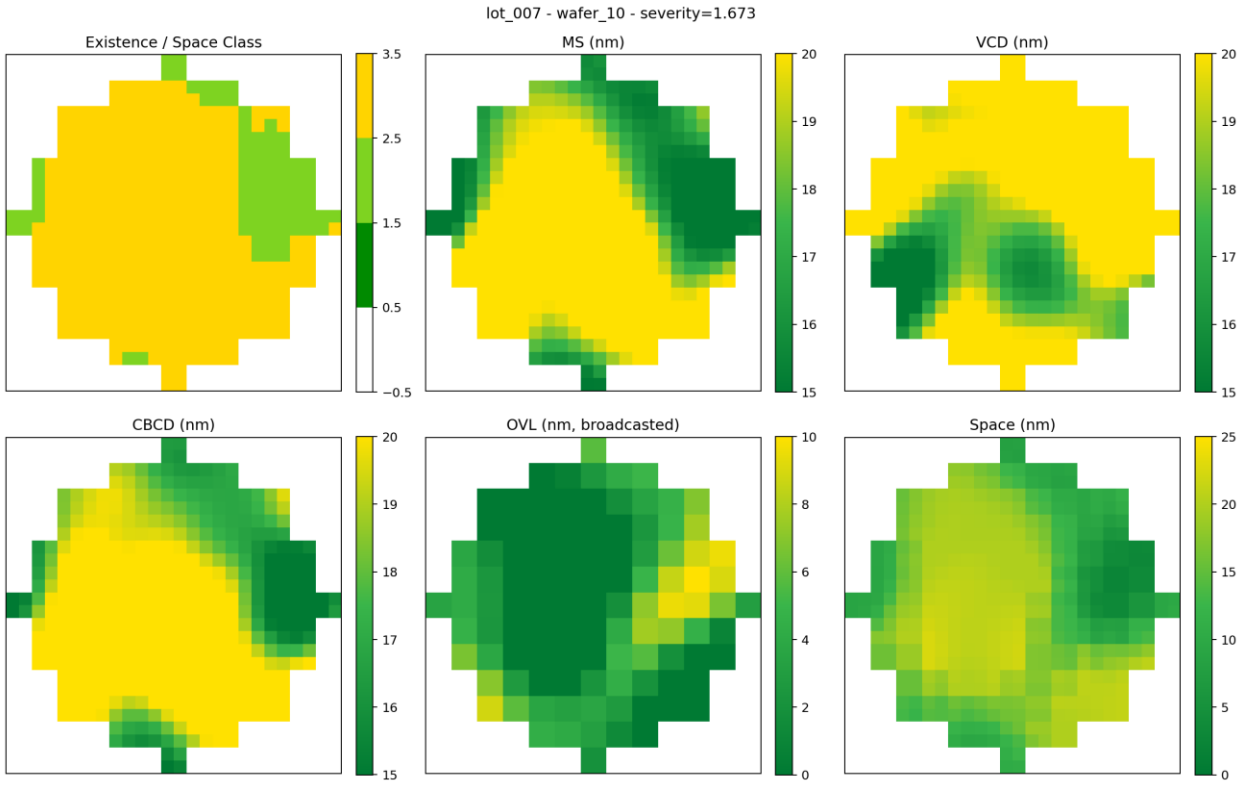


Figure 2: Example Candidate Wafer (lot 007 wafer 10 shown).

Outputs

There are two kinds of output, depending on whether you are **training** or running **inference**.

Inference outputs (per wafer)

When you run prediction on a test set, the engine mirrors the input folder tree under your chosen output directory, and writes the following inside each wafer's folder:

File	Contents
`ttf.csv`	A full 2-D grid of **continuous time-to-failure** predictions (one number per die).
`binary_label.csv`	A full 2-D grid of **good/bad decisions** (`1` = good, `0` = bad).
`wafer_maps.png`	A six-panel visual summary of the wafer (described below).

Background cells stay blank (`NaN`) in the CSVs, so the output grids line up cell-for-cell with the inputs.

The `wafer_maps.png` panel

This single image is a 2-row × 3-column dashboard for the wafer:

Position	Panel	What it shows
Top-left	**Space (V-L) spatial trend**	The via spacing, smoothly interpolated across the whole wafer (Kriging).
Top-middle	**MS (L-L) spatial trend**	The line spacing, smoothly interpolated across the whole wafer.
Top-right	**Predicted lifetime (TTF)**	Per-die lifetime, drawn as coloured tiles on a log colour scale (turbo colormap).
Bottom-left	**Ground-truth labels**	The true good/bad map from `ExistenceClass.csv` (cool = bad, warm = good).
Bottom-middle	**Predicted labels**	The engine's good/bad decision for each die. Compare against bottom-left to see where it agrees.
Bottom-right	*(intentionally blank)*	Hidden to keep the layout clean.

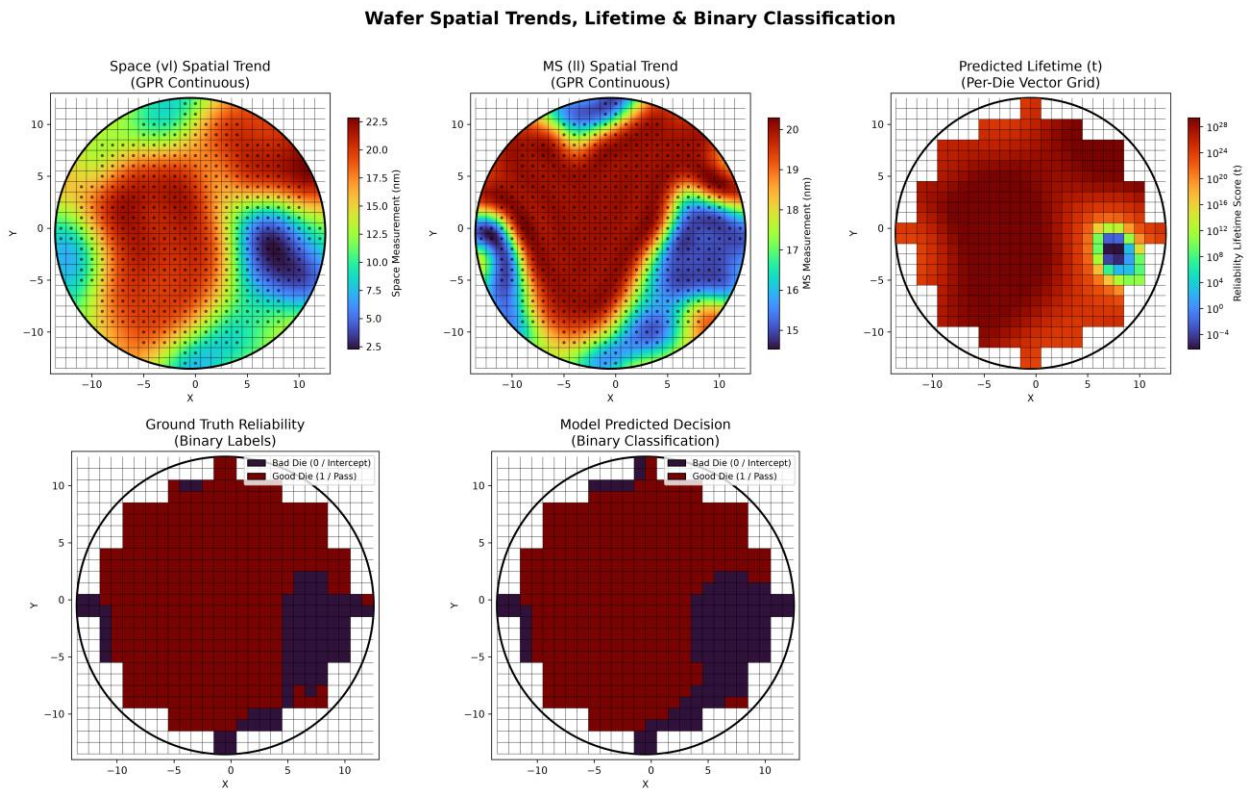


Figure 3: Inference Output (example shown is Power Law with Deltas).

Training outputs (per run)

When you train a model, the engine writes a small results folder (default `./configs/<name>/`) containing:

File	Contents
`config.json`	The **tuned settings.** The four numbers the engine found, plus which model family to use, plus any **trained deltas**. **This is the file you feed to inference.**
`metadata.json`	The full training log. every trial's settings, the train/validation scores, the best trial, the fitted delta values, and (when a held-out test set exists) the final test score.
`metric_curve.png`	A convergence plot of training-vs-validation score across the tuning trials.

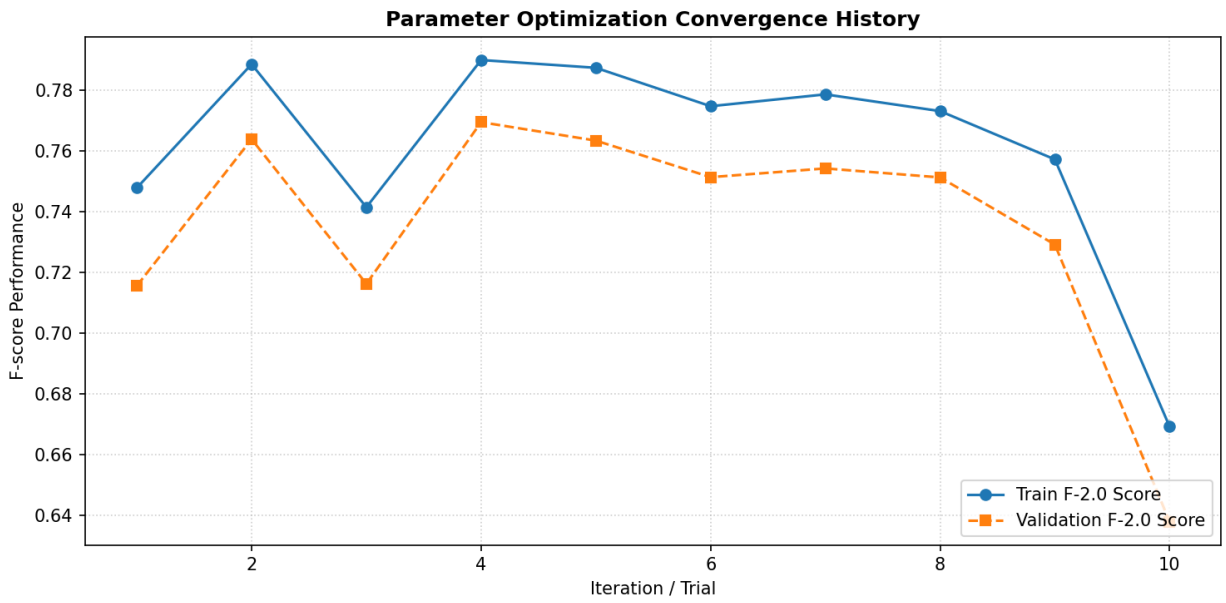


Figure 4: Matric Curve for GPR 2.0 with 10 trials.

A real `config.json` looks like this (GPR example):

```
{
  "pipeline_type": "GPR",
  "M_structures": 3600000,
  "F_target": 1.2087541473056965e-06,
  "N_samples_per_dim": 16,
  "unit_model_kwargs": { "model_type": "GPR", "actual_vl_max": 40.0, "actual_ll_max":
20.0 },
  "threshold": 1758.3429243407004
}
```

A delta-trained DPM config carries an extra `deltas` block inside `unit_model_kwargs`:

```
{
  "pipeline_type": "DPM",
  "M_structures": 2100000,
  "F_target": 3.4e-05,
  "N_samples_per_dim": 12,
  "unit_model_kwargs": {
    "model_type": "SqrtE",
    "deltas": { "m_val_delta_1": 0.31, "m_val_delta_2": -0.04,
                "Es_val_delta_1": 0.12, "Es_val_delta_2": 0.90,
                "gamma_val_delta_1": 8.7, "gamma_val_delta_2": 0.05 }
  },
  "threshold": 41.87
}
```

Project / File Structure

```
BEOL_TDDB_Physical_Models_Chris_Fork/   ← run everything from here (the project root)
│
├── create_split.py          ← STEP 1: builds the train/val/test split manifest
├── train.py                 ← STEP 2: trains/tunes a model, writes a config
├── inference.py             ← STEP 3: runs a trained config on new wafers
├── run_all_models.ps1       ← convenience: trains the model variants in one go
├── split_manifest.json      ← output of STEP 1 (the agreed data split)
├── requirements.txt         ← Python dependencies
│
├── configs/                 ← saved training results (one folder per model)
│   ├── GPR/  ├── DPM_SqrtE_deltas/  ├── DPM_PowerLaw_deltas/  └── ...
│   │   ├── config.json      ← feed THIS to inference.py
│   │   ├── metadata.json
│   │   └── metric_curve.png
│
└── src/                     ← the engine internals (you rarely touch these)
    ├── csv_utils.py                  ← reads/writes wafer CSVs, handles background
cells
    ├── prediction_pipeline.py        ← assembles Layers 1+2+3 from a config
    ├── multiprocessing_wrapper.py    ← runs many wafers in parallel
    ├── param_optimizer.py            ← the automatic tuning loop (Optuna) + delta
search
    │
    ├── unit_level_models/            ← LAYER 1
    │   ├── base_class.py             ← the trainable-parameter interface deltas hang
off
    │   ├── dpm_models.py             ← physics: PowerLaw / SqrtE / InverseE (+
deltas)
    │   ├── dpm_models_scaled.py      ← physics with spacing scale k
    │   ├── weibull_gpr_model.py      ← the GPR / ML surrogate
    │   ├── model_factory.py          ← picks the right model from its name
    │   └── weibull_gpr_model_simulated_data/   ← pre-computed simulation surfaces
    │
    ├── wafer_level_models/           ← LAYER 2
    │   ├── wafer_reliability_engines.py   ← Kriging + weakest-link solver
    │   └── linear_engine.py               ← the zero-physics baseline
    │
    ├── reliability_classifiers/      ← LAYER 3
    │   └── binary_classifier.py      ← threshold decision + scoring
    │
    └── plot_utils/                   ← the wafer_maps.png drawing code
```

How to Run the Flow

The modern flow has **three steps**: split → train → infer. If only predictions are desired, inference can be directly performed from a pretrained config.

Environment setup

Open **Windows PowerShell** in the project root and create a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install -r requirements.txt
```

The dependencies are: `numpy`, `matplotlib`, `scikit-learn`, `tqdm`, `optuna`. **No GPU is required.** The code automatically pins each worker to a single math thread (`OMP_NUM_THREADS=1` etc.)

Step 1 — Create the data split

Script: `create_split.py` → produces `split_manifest.json`

Before training you decide *which wafers teach the model, which wafers check it, and which wafers are held back for a final unbiased grade*. This is the **train / validation / test split**.

Notes on splitting:

- **Splitting happens at the LOT level, never the wafer level.** Wafers from the same lot are correlated (same process excursion, same tooling). If some wafers from a lot trained the model and others tested it, the test score would be inflated. Splitting whole lots prevents this leakage.
- **Lots are stratified by failure rate.** The lots are sorted by how many bad dies they contain, then dealt out across the three splits so each split spans the full range from clean lots to failure-heavy lots.

Run it:

```
python create_split.py --dataset-path data/
```

Useful options:

Argument	Default	Meaning
`--dataset-path`	`data/`	Root folder holding your `lot_*` directories.

Argument	Default	Meaning
`--train-ratio`	`0.6`	Fraction of lots used for training.
`--val-ratio`	`0.2`	Fraction used for validation (test gets the rest).
`--seed`	`42`	Random seed. Keep it fixed so the split is reproducible.
`--output`	`split_manifest.json`	Where to save the manifest.

The script prints a per-lot and per-split summary (lots, wafers, fail counts, failure rate) and saves `split_manifest.json`. That manifest is just a list of which wafer folders belong to `train`, `val`, and `test`. Training in Step 2.

Step 2 — Train the engine

Script: `train.py` → produces `configs/<name>/config.json` (+ metadata + curve)

Training is where the engine **tunes itself**. It searches for the best values of the control numbers and, if indicated, the physics deltas.

Training Flow

1. Reads the wafers named in your split manifest (train / val / test).
2. Runs **Bayesian optimisation** (the Optuna library, using a TPE sampler with a fixed seed of 42 for reproducibility) over **three** settings:

- `M_structures` — total equivalent structure count per die (search range 100 K – 5 M).
- `F_target` — target cumulative failure rate (search range 1e-6 – 1e-2, log scale).
- `N_samples_per_dim` — how densely each die is sub-sampled in space (1 – 16).
- *(with `--train-deltas`)* — **plus the physics deltas**, each searched inside its bounds.

1. For each trial, it predicts a TTF for every die, then **analytically finds the single best good/bad threshold** on the training set (a fast exact sweep over all candidate cut points).
2. It scores that threshold on the **validation** set with an **F-beta** score, and uses that score to steer the next trial.
3. After all trials, it saves the best combination (the three settings, any trained deltas, **plus** the resolved `threshold`) into `config.json`.

Command-line arguments

Argument	Required?	Default	Meaning
`--pipeline-type`	**Yes**	—	`GPR`, `DPM`, or `Linear`.
`--model-type`	DPM only	—	`PowerLaw`, `SqrtE`, `InverseE`, or a `...Scaled` variant. (Set automatically for `GPR` / `Linear`.)
`--spacing-scale`	Scaled DPM only	`1.0`	The divisor `k` for scaled models (e.g. `3.0`). Ignored by other models.
`--train-deltas`	No	off	**Also fit the physics deltas.** DPM only. Ignored by GPR/Linear.
`--delta-l2`	No	`0.0`	L2 shrink of deltas toward the physical baseline (`0`). A light value (e.g. `0.01`) prefers the smallest, most physically-plausible correction.
`--anchor-years`	No	`None`	Softly pull the learned decision boundary toward a physical lifetime (e.g. `10`) as an absolute-scale *prior*. Off by default.
`--anchor-weight`	No	`0.0`	Strength of that anchor prior. Off by default.
`--manifest`	recommended	—	Path to `split_manifest.json`. Overrides the two path options below.
`--training-path`	if no manifest	—	Root of the training dataset.
`--validation-path`	if no manifest	—	Root of the validation dataset.
`--fscore-beta`	No	`2.0`	F-beta weight (see How to Adjust).

Argument	Required?	Default	Meaning
`--n-trials`	No	`20`	How many optimisation trials to run. Use **more** when deltas are on (they add search dimensions). 40 is a good default.
`--batch-size`	No	`1`	Wafers handed to each worker at a time.
`--num-workers`	No	`4`	CPU cores to use.
`--save-path`	No	`./configs/<auto-name>/`	Where to write the results.

> **Two ways to point at data.** The recommended way is `--manifest split_manifest.json` (it carries train, val **and** test, and gives you the final held-out grade). The alternative way is to pass `--training-path` and `--validation-path` to two separate folders; this still works but has no held-out test stage.

The four levers of delta training

When `--train-deltas` is on, four controls shape the fit. Only the first is required; the rest are optional refinements:

Lever	Flag / location	Purpose
Bounds	in `dpm_models.py`	Hard limits on each delta.
Regularisation	`--delta-l2`	Prefer small corrections. Picks the most plausible, least deviated fit among equivalent ones.
Anchor	`--anchor-years` + `--anchor-weight`	A prior that ties the decision boundary to a real lifetime, restoring absolute units.
Objective	swappable in code	F-beta

Training examples

GPR (works out of the box):

```
python train.py --pipeline-type GPR `
  --manifest split_manifest.json `
  --num-workers 8 --n-trials 20
```

DPM — Inverse-E, unscaled (baseline):

```
python train.py --pipeline-type DPM --model-type InverseE `
  --manifest split_manifest.json `
  --num-workers 8 --n-trials 20
```

Delta-trained DPM — SqrtE (the recommended physics path):

```
python train.py --pipeline-type DPM --model-type SqrtE --train-deltas `
  --delta-l2 0.01 `
  --manifest split_manifest.json `
  --num-workers 20 --n-trials 40 `
  --save-path configs/DPM_SqrtE_deltas/
```

To add the absolute-scale anchor, append e.g. `--anchor-years 10 --anchor-weight 0.05`.

Linear baseline:

```
python train.py --pipeline-type Linear `
  --manifest split_manifest.json `
  --num-workers 8 --n-trials 20
```

Each run drops its `config.json`, `metadata.json`, and `metric_curve.png` into a `configs/...` sub-folder.

Train everything at once

To sweep the model variants against the same split, use the bundled PowerShell script:

```
.\run_all_models.ps1
```

Open the script first to set the constants at the top (`$N_TRIALS`, `$NUM_WORKERS`, and (for scaled models) `$SCALE`). It runs each model in turn and reports `[DONE]` / `[FAILED]` for each.

Step 3 — Run inference (prediction)

Script: `inference.py` → writes `ttf.csv`, `binary_label.csv`, `wafer_maps.png` per wafer

Inference takes a **trained config** and applies it to a folder of wafers, producing the prediction files and dashboards described in Outputs.

Command-line arguments

Argument	Required?	Meaning
`--config-path`	this **or** manual params	Path to a `config.json` from training. The easy path.
`--test-path`	**Yes**	Root of the wafers to predict on.
`--save-path`	**Yes**	Where to write the outputs (the input tree is mirrored here).
`--num-workers`	No	CPU cores (default 4).
`--batch-size`	No	Wafers per worker batch (default 1).
`--pipeline-type` / `--model-type` / `--M-structures` / `--F-target` / `--N-samples-per-dim` / `--threshold`	only if **not** using a config	Supply every setting by hand instead of from a file.

Recommended — use a trained config:

```
python inference.py `
  --pipeline-type GPR `
  --M-structures 3600000 --F-target 1.21e-6 `
  --N-samples-per-dim 16 --threshold 1758.34 `
  --manifest split_manifest.json --split test `
  --save-path output/test_manual/ `
  --num-workers 8
```

Manual settings (no config file):

```
python inference.py `
  --pipeline-type GPR `
  --M-structures 3600000 --F-target 1.21e-6 `
  --N-samples-per-dim 16 --threshold 1758.34 `
  --manifest split_manifest.json --split test `
  --save-path output/test_manual/ `
```

```
  --num-workers 8
```

> **NOTE: Scaled and delta-trained models: always use `--config-path`.** The manual flags on `inference.py` cannot express a scaling factor `k` or a set of deltas. Both live inside the `config.json` that training produced, so pointing at that file makes inference pick them up automatically.

How to Adjust

This section is the modern equivalent of the old `Global_Params.py` discussion. Most of these are settled for you by training but understanding them provides context and allows modification if needed.

The four core control numbers

Setting	Plain-English meaning	Effect of increasing it
`M_structures`	How many equivalent micro-structures a die is treated as containing. More structures = more chances for the weakest one to fail.	Predicts **shorter** lifetimes / more pessimistic.
`F_target`	The cumulative failure probability you are designing against (e.g. 1e-4 ≈ 100 ppm).	A looser target → **longer** predicted lifetimes.
`N_samples_per_dim`	How finely each die is sampled in space. A value of 16 means a 16×16 = 256-point grid inside every die.	More spatial detail and accuracy, but **more compute** (cost grows with the square).
`threshold`	The pass/fail cut on lifetime. TTF below it = bad.	A higher cut rejects more dies (stricter).

The first three feed Layers 1–2; the `threshold` is Layer 3. Training searches the first three and then solves the `threshold` exactly.

Manual Adjustment

Setting	Where	What it does
`--fscore-beta`	train	Balances catching bad dies (recall) against not over-rejecting good dies (precision). The engine treats **bad dies as the thing to catch**. `beta = 2.0` (default) leans toward **recall.** Simply, it would rather flag a questionable die than miss a real failure. Lower it toward `1.0` for a more balanced trade-off.
`--n-trials`	train	How hard the optimiser searches. `10` is quick; `20` is standard; use **`40`+ when `--train-deltas` is on**, since deltas add search dimensions.
`--train-deltas`	train	Turn on physics-delta calibration (DPM only). See delta training.
`--delta-l2`	train	How firmly to keep deltas small. `0.01` is a sensible light default.
`--anchor-years` / `--anchor-weight`	train	Optional prior tying the decision boundary to a real lifetime; restores absolute physical units.
`--spacing-scale` (k)	train	Start at `3.0` for 9–22 nm data; `1.0` disables it. Superseded by deltas.
`--num-workers`	train / infer	CPU cores. Throughput scales roughly linearly up to your physical core count. Start at `4`, raise to match your CPU.
`--batch-size`	train / infer	Wafers per worker hand-off. `1` is fine for almost everything.
`V_OP`	code (`dpm_models.py`, default 0.7 V)	Operating voltage for the physics models. Field is `V_OP / spacing`, so this scales the whole physics family.
`m_param`, `radius`	code (GPR loader)	Which pre-computed simulation surface the GPR model loads (`m_param` 1–10, `radius` 0.45 or 0.65).

Glossary

- **TTF** — Time-to-Failure. The predicted lifetime of a die (real units for physics models, simulation-derived for GPR, spacing-sum proxy for Linear).
- **TDDB** — Time-Dependent Dielectric Breakdown, the failure mechanism being modelled.
- **V-L / Space** — via-line spacing (the `Space.csv` measurement).
- **L-L / MS** — line-line spacing / metal stack (the `MS.csv` measurement).
- **Weibull β (slope) / η (scale)** — the two numbers that describe a structure's failure-time distribution; Layer 1's output.
- **Weakest-link** — the rule that a die fails when its weakest internal structure fails; how Layer 2 combines many structures into one die lifetime.
- **Kriging** — spatial Gaussian-process interpolation used to fill in spacing values between measured points.
- **F-beta score** — a single accuracy number that blends recall and precision, weighted by `beta`; the target that training maximises.
- **Optuna / TPE** — the Bayesian optimisation library and sampler that drive the automatic tuning loop.
- **Spacing scale `k`** — the older one-parameter fix that divides measured spacing before the physics runs, re-centring wide spacings into the calibrated band.
- **Deltas** — small, bounded, additive corrections placed inside the DPM physics equations; training fits them (with `--train-deltas`) to re-shape the model to your process. All default to 0, where the model is identical to the original IMEC fit.
- **Anchor** — an optional prior (`--anchor-years`) that ties the learned decision boundary to a real lifetime, restoring absolute physical units to a rank-based fit.
- **Regularisation (`--delta-l2`)** — a light pull keeping deltas small, so the most physically-plausible of several equivalent fits is chosen.
- **Linear baseline** — a zero-physics model whose TTF is just `VL + LL` spacing; used to measure how much the physics machinery is actually worth.
- **TSK-Alpha** — the assumed single-test-structure-per-cell geometry this tool models.
